Number Theory for Contests

Expanded reference: every notation explained, every tool with a snippet

Read §0 once. In a contest, jump via the table of contents or the "Recognize the problem" index in §13.

Contents

0 Notation: What Every Symbol Means **3**

1 Primes and Factorization **3**

1.1 Linear sieve: primes, smallest prime factor, φ, μ in $O(N)$ . . . 4
1.2 Factorizing with spf in $O(\log n)$ . . . 4
1.3 Trial division for one $n \le 10^{12}$–10^{14}, $O(\sqrt{n})$ . . . 4
1.4 From factorization to everything . . . 4
1.5 Useful parity / shape facts . . . 5

2 Multiplicative Function Catalog **5**

2.1 Jordan totient in one line . . . 5
2.2 Generic linear sieve for any multiplicative f . . . 5

3 The Four Moves (the whole game) **6**

4 Dirichlet Convolution: The Algebra **7**

4.1 Identity table (left: convolution, right: what it says in plain sums) . . . 7
4.2 How to find a convolution yourself (no series needed): match on prime powers . . . 7

5 Möbius Inversion and Fast Transforms **7**

5.1 The two directions . . . 7
5.2 Fast transforms in $O(N \log\log N)$ (one side is $\mathbf{1}$ or μ) . . . 8
5.3 Inclusion–exclusion over prime factors (small ω) . . . 8

6 Floor Blocks and the Hyperbola Method **8**

6.1 Floor blocks . . . 8
6.2 Collecting on $T = de$ . . . 9
6.3 Dirichlet hyperbola method . . . 9
6.4 `floor_sum`: $\sum_{i=0}^{n-1} \lfloor (ai+b)/m \rfloor$ in $O(\log m)$ . . . 9

7 Prefix Sums at Huge n (10^9–10^{11}) **9**

7.1 Closed forms in $O(\sqrt{n})$, no recurrence . . . 9
7.2 Du's sieve ("Dujiao") — $\Phi(n)$, $M(n)$ for $n \le 10^{10}$ in $O(n^{2/3})$ . . . 10
7.3 When Du fails: powerful numbers and Min_25 . . . 10

8 GCD, LCM and Divisor Sums **10**

8.1 Array gcd template . . . 11
8.2 Counting under gcd conditions — subsequences . . . 11
8.3 Necklaces (Burnside + classification) . . . 11

9 Modular Arithmetic **11**

9.1 Core snippets . . . 12
9.2 Theorem table . . . 12
9.3 CRT, general (non-coprime moduli) . . . 12

9.4 Miller–Rabin + Pollard Rho (factor up to 10^{18}) . . . 13
9.5 Order, primitive roots, discrete roots . . . 13
9.6 Quadratic residues . . . 13
9.7 Lifting the Exponent (LTE) . . . 13
9.8 Extra medium-level modular facts . . . 14

10 Combinatorial Number Theory Cheats **14**

11 Sieves, Costs and Sizing **14**

12 Workflow and Debugging Checklist **15**

13 Recognize the Problem → Jump Here **15**

0. Notation: What Every Symbol Means

Read this section once before the contest. After that, every formula below should read like plain English.

Symbol	Meaning (in words) and tiny example
$d \mid n$	"d divides n". $3 \mid 12$ is true, $5 \mid 12$ false.
$\sum_{d\mid n}$	Sum over every *positive divisor* d of n. For $n=6$: $d \in \{1,2,3,6\}$.
$[P]$	Iverson bracket: 1 if statement P is true, else 0. $[\gcd(i,j)=1]$ is "are they coprime?"
$\lfloor x \rfloor$, $\lfloor \frac{n}{d} \rfloor$	Floor. In C++, `n/d` for positive integers. $\lfloor \frac{n}{d} \rfloor$ = how many multiples of d are in $[1,n]$.
$p^k \parallel n$	p^k divides n exactly (p^{k+1} does not).
$v_p(n)$	Exponent of prime p in n. $v_2(24)=3$.
$n=\prod p_i^{e_i}$	Prime factorization of n. $360 = 2^3 \cdot 3^2 \cdot 5$.
$\omega(n)$	Number of *distinct* primes. $\omega(360)=3$.
$\Omega(n)$	Number of primes *with multiplicity*. $\Omega(360)=3+2+1=6$.
$\mathrm{rad}(n)$	Product of distinct primes. $\mathrm{rad}(360)=30$.
$d(n)$ or $\sigma_0(n)$	Number of divisors. $d(12)=6$.
$\sigma(n)$, $\sigma_k(n)$	Sum of divisors; sum of k-th powers of divisors. $\sigma(12)=28$.
$\varphi(n)$	Euler phi: $\#\{1 \le i \le n : \gcd(i,n)=1\}$. $\varphi(12)=4$ $(1,5,7,11)$.
$\mu(n)$	Möbius: 0 if a square >1 divides n; else $(-1)^{\#\text{primes}}$. $\mu(1)=1, \mu(6)=1, \mu(5)=-1, \mu(12)=0$.
$\mu^2(n)$	1 if n is squarefree, else 0.
$\lambda(n)$	Liouville: $(-1)^{\Omega(n)}$. $\lambda(12)=(-1)^3=-1$.
$\varepsilon(n)$	"Identity" function: 1 if $n=1$ else 0. Same as $[n=1]$.
$\mathbf{1}(n)$	Constant function 1.
$\mathrm{Id}(n)$, $\mathrm{Id}_k(n)$	n and n^k (as functions).
$J_k(n)$	Jordan totient: number of k-tuples in $[1,n]^k$ whose gcd with n is 1. $J_1=\varphi$.
$\psi(n)$	Dedekind psi: $n\prod_{p\mid n}(1+1/p)$.
$\Lambda(n)$	von Mangoldt: $\ln p$ if $n=p^k$, else 0. Rare in CP.
$F(n)$ (capital)	Prefix sum of lowercase f: $F(n)=\sum_{i=1}^{n} f(i)$. So $\Phi(n)=\sum_{i\le n}\varphi(i)$, $M(n)=\sum_{i\le n}\mu(i)$.
$(f*g)(n)$	**Dirichlet convolution**: $\sum_{d\mid n} f(d)\,g(n/d)$ — "split n into two factors $d\cdot\frac{n}{d}$, multiply, add up".
$a \equiv b \pmod m$	$m \mid (a-b)$.
$a^{-1} \bmod m$	The x with $ax \equiv 1 \pmod m$ (exists iff $\gcd(a,m)=1$).
$\mathrm{ord}_m(a)$	Smallest $k \ge 1$ with $a^k \equiv 1 \pmod m$.
$\binom{n}{r}$	Binomial coefficient.

How to read a convolution like $\varphi * \mathbf{1} = \mathrm{Id}$

Plug the definition: $(\varphi*\mathbf{1})(n)=\sum_{d\mid n}\varphi(d)\cdot\mathbf{1}(n/d)=\sum_{d\mid n}\varphi(d)$, and the identity says this equals $\mathrm{Id}(n)=n$. Every "$f*g=h$" line in this document is just "$\sum_{d\mid n} f(d)g(n/d)=h(n)$". Whenever one side is $\mathbf{1}$, it is simply "sum of f over divisors".

Multiplicative means $f(ab)=f(a)f(b)$ whenever $\gcd(a,b)=1$ (and $f(1)=1$). Then f is fixed by its values on prime powers:

$$f(p_1^{e_1}\cdots p_r^{e_r}) = f(p_1^{e_1})\cdots f(p_r^{e_r}).$$

Completely multiplicative: $f(ab)=f(a)f(b)$ for *all* a,b (e.g. n^k, λ).

1. Primes and Factorization

1.1 Linear sieve: primes, smallest prime factor, φ, μ in $O(N)$

Every composite $i \cdot p$ is crossed out exactly once, by its smallest prime p. Use for $N \le 10^7$ (memory: `int spf[N]` is 40MB at 10^7; use $\le 2 \cdot 10^7$ carefully).

```
const int N = 1e7 + 5;
int spf[N], phi[N], mu[N]; vector<int> primes;
void sieve() {
    phi[1] = mu[1] = 1;
    for (int i = 2; i < N; i++) {
        if (!spf[i]) { spf[i] = i; primes.push_back(i); phi[i] = i - 1; mu[i] = -1; }
        for (int p : primes) {
            if (p > spf[i] || (long long)i * p ≥ N) break;
            spf[i * p] = p;
            if (i % p == 0) { phi[i * p] = phi[i] * p; mu[i * p] = 0; break; }
            phi[i * p] = phi[i] * (p - 1); mu[i * p] = -mu[i];
        }
    }
}
```

Rule behind the two branches: if $p \mid i$, then p is an *existing* prime of i: φ multiplies by p, μ becomes 0. If $p \nmid i$, p is *new*: φ multiplies by $p-1$, μ flips sign.

1.2 Factorizing with spf in $O(\log n)$

```
vector<pair<int,int>> factor(int n) {           // needs spf[] up to n
    vector<pair<int,int>> f;
    while (n > 1) { int p = spf[n], e = 0; while (n % p == 0) n /= p, e++; f.push_back({p, e}); }
    return f;
}
```

1.3 Trial division for one $n \le 10^{12}$–10^{14}, $O(\sqrt{n})$

```
vector<pair<long long,int>> factorTrial(long long n) {
    vector<pair<long long,int>> f;
    for (long long p = 2; p * p ≤ n; p++) if (n % p == 0) {
        int e = 0; while (n % p == 0) n /= p, e++; f.push_back({p, e});
    }
    if (n > 1) f.push_back({n, 1});             // leftover is a prime
    return f;
}
```

Faster: loop only over sieved primes up to $\sqrt{n}$ ($\approx$ 78,000 primes below 10^6). For many queries up to 10^{18}: Miller–Rabin + Pollard Rho (§9.4).

1.4 From factorization to everything

Let $n = \prod p_i^{e_i}$. Then directly:

$$d(n) = \prod (e_i + 1), \qquad \sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}, \qquad \varphi(n) = n \prod \left(1 - \frac{1}{p_i}\right),$$

$$\prod_{d \mid n} d = n^{d(n)/2}, \qquad \#\{\text{squarefree divisors}\} = 2^{\omega(n)}, \qquad \#\{(a,b) : \operatorname{lcm}(a,b) = n\} = \prod (2e_i + 1).$$

```
long long phiOf(long long n) {                  // phi of one number, O(sqrt n)
    long long r = n;
    for (long long p = 2; p * p ≤ n; p++) if (n % p == 0) { while (n % p == 0) n /= p; r -= r / p; }
    if (n > 1) r -= r / n;
    return r;
}
```

Enumerate all divisors from the factorization (useful because $d(n) \le 1344$ for $n \le 10^9$):

```
vector<long long> divs = {1};
for (auto [p, e] : f) {
```

```
    int sz = divs.size(); long long pk = 1;
    for (int k = 1; k ≤ e; k++) { pk *= p; for (int i = 0; i < sz; i++) divs.push_back(divs[i] * pk); }
}
```

1.5 Useful parity / shape facts

- $d(n)$ is odd $\iff$ n is a perfect square. (Light bulb problems: bulb n toggled $d(n)$ times.)
- $\sigma(n)$ is odd $\iff$ n is a square or twice a square.
- $\varphi(n)$ is even for $n > 2$. $\varphi(n) = n - 1 \iff n$ prime.
- $\gcd(a,b) \cdot \operatorname{lcm}(a,b) = ab$. $\gcd(a,b) = \gcd(a, b-a) = \gcd(a, b \bmod a)$.
- $\gcd(F_m, F_n) = F_{\gcd(m,n)}$ (Fibonacci), and $\gcd(a^m - 1, a^n - 1) = a^{\gcd(m,n)} - 1$.
- Squarefree part: every n is uniquely $n = s \cdot t^2$ with s squarefree. Two numbers multiply to a square $\iff$ same squarefree part.
- $\lceil a/b \rceil = \lfloor (a + b - 1)/b \rfloor$ for $a \geq 0, b > 0$.

2. Multiplicative Function Catalog

Each function is determined by its value at p^k. That column is exactly what you feed into a multiplicative sieve (§2.2).

f	$f(p^k)$	Closed form for n	What it counts / when you see it
ε	$[k = 0]$	$[n = 1]$	The "neutral" function for convolution
$\mathbf{1}$	1	1	Constant; $f * \mathbf{1}$ = sum over divisors
Id_k	p^{jk} at p^j	n^k	Powers
μ	$1, -1, 0, 0, \ldots$	$(-1)^{\omega(n)}$ if squarefree else 0	Inclusion–exclusion over primes
φ	$p^k - p^{k-1}$	$n \prod_{p\mid n}(1 - \frac{1}{p})$	Coprime residues
$d = \sigma_0$	$k + 1$	$\prod(e_i + 1)$	Number of divisors
σ	$\frac{p^{k+1}-1}{p-1}$	$\prod \frac{p_i^{e_i+1}-1}{p_i-1}$	Sum of divisors
σ_j	$\frac{p^{j(k+1)}-1}{p^j-1}$	product of those	Sum of j-th powers of divisors
J_k	$p^{jk} - p^{(j-1)k}$	$n^k \prod_{p\mid n}(1 - p^{-k})$	k-tuples coprime to n (as a set)
ψ	$p^k + p^{k-1}$	$n \prod_{p\mid n}(1 + \frac{1}{p})$	Dedekind psi
μ^2	$[k \leq 1]$	$[n$ squarefree$]$	Squarefree indicator
2^ω	2 $(k \geq 1)$	$2^{\omega(n)}$	Squarefree divisors; coprime pairs (a,b) with $ab = n$
λ	$(-1)^k$	$(-1)^{\Omega(n)}$	Detects squares (sum over divisors)
rad	p $(k \geq 1)$	$\prod_{p\mid n} p$	Product of distinct primes
ω, Ω	1, k	—	**Not** multiplicative (additive: $f(ab) = f(a) + f(b)$)

To check if a function from a problem is multiplicative: test $f(ab) = f(a)f(b)$ on coprime a, b like $(2,3), (4,9), (4,3)$ with a brute force.

2.1 Jordan totient in one line

$J_k(n) = \#\{(a_1, \ldots, a_k) \in [1,n]^k : \gcd(a_1, \ldots, a_k, n) = 1\}$, and the key identity

$$\sum_{d\mid n} J_k(d) = n^k \qquad \text{(generalizes } \textstyle\sum_{d\mid n} \varphi(d) = n\text{)}.$$

2.2 Generic linear sieve for any multiplicative f

Track `pk[i]` = the full power of the smallest prime in i. Then $f(i) = f(i/pk) \cdot f(pk)$.

```cpp
// fill fpk(p, k, pk) = f(p^k).  Example below: sigma (sum of divisors).
long long fpk(long long p, int k, long long pk) { return (pk * p - 1) / (p - 1); }
int lp[N], cnt[N]; long long pk[N], f[N]; vector<int> pr;
void multSieve() {
    f[1] = 1;
    for (int i = 2; i < N; i++) {
        if (!lp[i]) { lp[i] = i; pr.push_back(i); pk[i] = i; cnt[i] = 1; f[i] = fpk(i, 1, i); }
        for (int p : pr) {
            if (p > lp[i] || (long long)i * p ≥ N) break;
            int x = i * p; lp[x] = p;
            if (i % p == 0) { pk[x] = pk[i] * p; cnt[x] = cnt[i] + 1;
                f[x] = (pk[x] == x) ? fpk(p, cnt[x], pk[x]) : f[x / pk[x]] * f[pk[x]]; break; }
            pk[x] = p; cnt[x] = 1; f[x] = f[i] * f[p];
        }
    }
}
```

Simpler $O(N \log N)$ alternative when f is a divisor sum: `for d:  for multiples m of d:  g[m] += f[d]*h[m/d]`.

3. The Four Moves (the whole game)

Almost every "sum of gcd/lcm/coprime" problem is solved by applying these four rewrites in order.

Move	Rewrite	When to use
1. Expand a value	$n = \sum_{d\mid n} \varphi(d)$, $n^k = \sum_{d\mid n} J_k(d)$	A *value* like $\gcd(i,j)$ appears in the sum
2. Expand a condition	$[n = 1] = \sum_{d\mid n} \mu(d)$, $[n \text{ is square}] = \sum_{d\mid n} \lambda(d)$	A *condition* like $\gcd(i,j) = 1$ appears
3. Swap the order	$\sum_{i\le n}\sum_{d\mid i} \to \sum_{d\le n}\sum_{i\le n,\ d\mid i}$	Always, immediately after move 1 or 2
4. Collect on $T = de$	$\sum_d \sum_e (\cdot)\left\lfloor \frac{n}{de}\right\rfloor \to \sum_T \left\lfloor \frac{n}{T}\right\rfloor h(T)$	Two nested divisor-sums remain

Two counting facts make move 3 pay off:

$$\#\{i \le n : d \mid i\} = \left\lfloor \frac{n}{d} \right\rfloor, \qquad \left\lfloor \frac{\lfloor n/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor.$$

And the **classification lemma**: group i by $g = \gcd(i,n)$; exactly $\varphi(n/g)$ values of $i \le n$ have $\gcd(i,n) = g$. So

$$\sum_{i=1}^{n} f(\gcd(i,n)) = \sum_{d\mid n} f(d)\,\varphi(n/d).$$

Full walk-through: $\sum_{i\le n}\sum_{j\le m}[\gcd(i,j)=1]$

Move 2: $[\gcd(i,j)=1] = \sum_{d\mid \gcd(i,j)} \mu(d)$.

Move 3: $d \mid \gcd(i,j)$ means $d \mid i$ and $d \mid j$. Pull d out: $\displaystyle\sum_d \mu(d)\cdot \#\{i \le n : d \mid i\} \cdot \#\{j \le m : d \mid j\} = \sum_{d\le \min(n,m)} \mu(d)\left\lfloor \frac{n}{d}\right\rfloor \left\lfloor \frac{m}{d}\right\rfloor$.

Compute: prefix-sum μ, use floor blocks (§6.1) $\Rightarrow O(\sqrt{n})$ per query.

Using move 1: $\sum_{i\le n}\sum_{j\le n}\gcd(i,j)$

$\gcd(i,j) = \sum_{d\mid\gcd(i,j)} \varphi(d)$. Swap: $= \sum_{d\le n} \varphi(d)\left\lfloor \frac{n}{d}\right\rfloor^2$. Done.

4. Dirichlet Convolution: The Algebra

$(f * g)(n) = \sum_{d|n} f(d)g(n/d)$. Rules: commutative, associative, ε is the identity ($f * \varepsilon = f$), distributes over $+$. Multiplicative $*$ multiplicative = multiplicative. If $f(1) \neq 0$, f has an inverse f^{-1} with $f * f^{-1} = \varepsilon$ (inverse of $\mathbf{1}$ is μ).

Pull-out rule (for completely multiplicative h, e.g. $h = \mathrm{Id}_k$): $h \cdot (f * g) = (h \cdot f) * (h \cdot g)$, where $\cdot$ is pointwise product. Example: from $\mu * \mathbf{1} = \varepsilon$ we get $(\mathrm{Id} \cdot \mu) * \mathrm{Id} = \varepsilon$.

4.1 Identity table (left: convolution, right: what it says in plain sums)

Conv.	Reads as	Conv.	Reads as
$\mu * \mathbf{1} = \varepsilon$	$\sum_{d\|n} \mu(d) = [n = 1]$	$\varphi * \sigma = \mathrm{Id} \cdot d$	$\sum_{d\|n} \varphi(d)\sigma(\frac{n}{d}) = n\, d(n)$
$\varphi * \mathbf{1} = \mathrm{Id}$	$\sum_{d\|n} \varphi(d) = n$	$\mathrm{Id}_k * \mu = J_k$	$\sum_{d\|n} \mu(d)(\frac{n}{d})^k = J_k(n)$
$\mu * \mathrm{Id} = \varphi$	$\varphi(n) = \sum_{d\|n} \mu(d)\frac{n}{d}$	$\mathrm{Id} * \mu^2 = \psi$	Dedekind psi
$\mathbf{1} * \mathbf{1} = d$	$d(n) = \sum_{d\|n} 1$	$\mathbf{1} * \mu^2 = 2^\omega$	$\sum_{d\|n} \mu^2(d) = 2^{\omega(n)}$
$\mathrm{Id} * \mathbf{1} = \sigma$	$\sigma(n) = \sum_{d\|n} d$	$\mathbf{1} * \lambda = [\square]$	$\sum_{d\|n} \lambda(d) = [n \text{ square}]$
$\mathrm{Id}_j * \mathbf{1} = \sigma_j$	$\sigma_j(n) = \sum_{d\|n} d^j$	$\lambda * \mu^2 = \varepsilon$	inverse of μ^2 is λ
$\mu * d = \mathbf{1}$	$\sum_{d\|n} \mu(d)d(\frac{n}{d}) = 1$	$\mathbf{1} * 2^\omega = d(n^2)$	$d(n^2) = \sum_{d\|n} 2^{\omega(d)}$
$\mu * \sigma = \mathrm{Id}$	$\sum_{d\|n} \mu(d)\sigma(\frac{n}{d}) = n$	$d * 2^\omega = d^2$	$d(n)^2 = \sum_{d\|n} d(\frac{n}{d})2^{\omega(d)}$
$\varphi * d = \sigma$	$\sum_{d\|n} \varphi(d)d(\frac{n}{d}) = \sigma(n)$	$\mathbf{1} * \Lambda = \ln$	$\sum_{d\|n} \Lambda(d) = \ln n$

Extra non-convolution identities:

$$\sum_{d|n} \frac{\mu^2(d)}{\varphi(d)} = \frac{n}{\varphi(n)}, \quad \sum_{d|n} \mu(d)\, d = \prod_{p|n}(1-p), \quad \sum_{d|n} \frac{\mu(d)}{d} = \frac{\varphi(n)}{n}, \quad d(ij) = \sum_{x|i}\sum_{y|j}[\gcd(x,y) = 1].$$

4.2 How to find a convolution yourself (no series needed): match on prime powers

Because both sides are multiplicative, **an identity $f * g = h$ is true iff it is true at every p^k**. At p^k the convolution is just

$$(f * g)(p^k) = \sum_{j=0}^{k} f(p^j)\, g(p^{k-j}).$$

Check $\varphi * \mathbf{1} = \mathrm{Id}$

At p^k: $\sum_{j=0}^{k} \varphi(p^j) = 1 + (p-1) + (p^2-p) + \cdots + (p^k - p^{k-1}) = p^k$. Telescopes. True.

Find g such that $\mu^2 = \mathbf{1} * g$

Compute $g = \mu^2 * \mu$ at prime powers: $g(p) = \mu^2(p) - 1 = 0$, $g(p^2) = \mu^2(p^2) - \mu^2(p) = -1$, $g(p^3) = 0$, ... So $g(n) = \mu(m)$ if $n = m^2$, else 0. Hence $\mu^2(n) = \sum_{d^2|n} \mu(d)$. Swap order $\Rightarrow$ #squarefree $\leq n = \sum_{d \leq \sqrt{n}} \mu(d) \left\lfloor \frac{n}{d^2} \right\rfloor$.

Inverse at prime powers: $f^{-1}(1) = 1$, $f^{-1}(p^k) = -\sum_{j=1}^{k} f(p^j) f^{-1}(p^{k-j})$. Use this when you need the "helper" g for Du sieve or a powerful-number trick.

5. Möbius Inversion and Fast Transforms

5.1 The two directions

- **Divisor form:** $g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d)\, g(n/d)$.
- **Multiple form** (more common in CP): $g(n) = \sum_{n|m} f(m) \iff f(n) = \sum_{n|m} \mu(m/n)\, g(m)$.

The CP pattern

"Count pairs with gcd *exactly* d" is hard; "count pairs where both are *divisible by* d" is easy ($= c(d)^2$ or $\left\lfloor \frac{n}{d} \right\rfloor^2$). Compute the easy one for all d, then convert to "exactly" by subtracting multiples, going from large d to small.

```
// divisible-by -> exactly   (O(N log N)). cnt[d] = #objects whose gcd is a multiple of d
for (int d = N; d ≥ 1; d--) for (int k = 2 * d; k ≤ N; k += d) cnt[d] -= cnt[k];
// exactly -> divisible-by
for (int d = 1; d ≤ N; d++) for (int k = 2 * d; k ≤ N; k += d) cnt[d] += cnt[k];
```

You never write μ here, so you cannot get a sign wrong.

Classic: number of pairs $i < j$ with $\gcd(a_i, a_j) = 1$, $a_i \le 10^6$

`c[d]` = how many a_i divisible by d (loop over multiples of d in a frequency array). Pairs with gcd divisible by d: `f[d] = c[d]*(c[d]-1)/2`. Subtract-downward. Answer: `f[1]`. Equivalently $\sum_d \mu(d)\binom{c(d)}{2}$.

5.2 Fast transforms in $O(N \log\log N)$ (one side is 1 or μ)

Like a prefix sum done prime-by-prime (each prime is one "dimension" of exponents).

```
for (int p : primes) for (int i = 1; (long long)i * p ≤ N; i++) f[i * p] += f[i]; // f = f*1   (sum over
    divisors)
for (int p : primes) for (int i = N / p; i ≥ 1; i--) f[i * p] -= f[i];           // f = f*mu  (undo
    above)
for (int p : primes) for (int i = N / p; i ≥ 1; i--) f[i] += f[i * p];           // sum over multiples
for (int p : primes) for (int i = 1; (long long)i * p ≤ N; i++) f[i] -= f[i * p]; // undo multiples-sum
```

5.3 Inclusion–exclusion over prime factors (small ω)

Count $i \le X$ coprime to n: take the distinct primes of n (at most 15 for $n \le 10^{18}$), loop subsets.

$$\#\{i \le X : \gcd(i,n) = 1\} = \sum_{d \mid \mathrm{rad}(n)} \mu(d) \left\lfloor \frac{X}{d} \right\rfloor.$$

```
long long coprimeCount(long long X, vector<long long>& ps) {   // ps = distinct primes of n
    int k = ps.size(); long long res = 0;
    for (int mask = 0; mask < (1 << k); mask++) {
        long long d = 1; int bits = 0;
        for (int i = 0; i < k; i++) if (mask >> i & 1) d *= ps[i], bits++;
        res += (bits & 1 ? -1 : 1) * (X / d);
    }
    return res;
}
```

Sum of those i instead of count: replace `X/d` by $d \cdot \frac{q(q+1)}{2}$ with $q = \lfloor \frac{X}{d} \rfloor$. "$k$-th number coprime to n": binary search on X.

6. Floor Blocks and the Hyperbola Method

6.1 Floor blocks

$\lfloor \frac{n}{i} \rfloor$ takes only $\le 2\sqrt{n}$ distinct values, in contiguous runs of i. If the current run starts at l, it ends at $r = \left\lfloor \frac{n}{\lfloor n/l \rfloor} \right\rfloor$.

```
for (long long l = 1, r; l ≤ n; l = r + 1) {
    long long q = n / l; r = n / q;        // q is constant for i in [l, r]
    ans += (F(r) - F(l - 1)) * g(q);       // F = prefix sum of f
}
// two numbers n, m: l ≤ min(n,m), r = min(n/(n/l), m/(m/l))
// floor(n/i) for i in [1, k] only: also cap r = min(r, k)
```

Master pattern: $\sum_{i=1}^{n} f(i)\, g\left(\left\lfloor \frac{n}{i} \right\rfloor\right) = \sum_{\text{blocks } [l,r]} \left(F(r) - F(l-1)\right) g\left(\left\lfloor \frac{n}{l} \right\rfloor\right)$ in $O(\sqrt{n})$ if F is $O(1)$.

Common floor-block bugs

- $n \bmod i$ appears? Use $n \bmod i = n - i\lfloor \frac{n}{i} \rfloor$, so $\sum_{i \le n}(n \bmod i) = n^2 - \sum_i i \lfloor \frac{n}{i} \rfloor$.
- Ceil blocks: $\lceil n/i \rceil = \lfloor (n-1)/i \rfloor + 1$; block on $n-1$.

- Overflow: $F(r)$ often needs mod or `__int128`.

6.2 Collecting on $T = de$

Given $\sum_d w(d) \sum_e \mu(e) \left\lfloor \frac{n}{de} \right\rfloor \left\lfloor \frac{m}{de} \right\rfloor$, put $T = de$:

$$= \sum_{T \le \min(n,m)} \left\lfloor \frac{n}{T} \right\rfloor \left\lfloor \frac{m}{T} \right\rfloor h(T), \qquad h(T) = \sum_{d \mid T} w(d)\mu(T/d) = (w * \mu)(T).$$

Sieve h once (linear sieve or the $O(N \log\log N)$ μ-transform), prefix-sum it; each query is one block loop. This is how multi-query gcd sums (e.g. $\sum\sum[\gcd(i,j) \text{ is prime}]$) are done.

$\sum_{i \le n} \sum_{j \le m} [\gcd(i,j)$ prime$]$, 10^4 queries

$= \sum_p \sum_e \mu(e) \left\lfloor \frac{n}{pe} \right\rfloor \left\lfloor \frac{m}{pe} \right\rfloor$. With $T = pe$: $h(T) = \sum_{p \mid T,\ p \text{ prime}} \mu(T/p)$. Precompute: `for each prime p, for multiples T of p: h[T] += mu[T/p]`.

6.3 Dirichlet hyperbola method

If F and G (prefix sums of f, g) are cheap, then for $s = \lfloor \sqrt{n} \rfloor$:

$$\sum_{i \le n} (f * g)(i) = \sum_{a \le s} f(a)\, G\left(\left\lfloor \frac{n}{a} \right\rfloor\right) + \sum_{b \le s} g(b)\, F\left(\left\lfloor \frac{n}{b} \right\rfloor\right) - F(s)\, G(s).$$

Intuition: count lattice points (a, b) with $ab \le n$ weighted by $f(a)g(b)$; split into $a \le s$ or $b \le s$, subtract the double-counted square.

6.4 `floor_sum`: $\sum_{i=0}^{n-1} \lfloor (ai + b)/m \rfloor$ in $O(\log m)$

Use when the numerator is *linear in* i (lattice points under a line), not n/i.

```
long long floor_sum(long long n, long long m, long long a, long long b) { // a,b ≥ 0
    long long ans = 0;
    while (true) {
        if (a ≥ m) { ans += (n - 1) * n / 2 * (a / m); a %= m; }
        if (b ≥ m) { ans += n * (b / m); b %= m; }
        long long y = a * n + b; if (y < m) break;
        n = y / m; b = y % m; swap(m, a);
    }
    return ans;
}
```

7. Prefix Sums at Huge n (10^9–10^{11})

7.1 Closed forms in $O(\sqrt{n})$, no recurrence

$$\sum_{i \le n} d(i) = \sum_{i \le n} \left\lfloor \frac{n}{i} \right\rfloor = 2 \sum_{i \le \sqrt{n}} \left\lfloor \frac{n}{i} \right\rfloor - \lfloor \sqrt{n} \rfloor^2, \qquad \sum_{i \le n} \sigma(i) = \sum_{i \le n} i \left\lfloor \frac{n}{i} \right\rfloor \text{ (block it)},$$

$$\#\{\text{squarefree} \le n\} = \sum_{d \le \sqrt{n}} \mu(d) \left\lfloor \frac{n}{d^2} \right\rfloor, \qquad \#\{k\text{-free} \le n\} = \sum_{d \le n^{1/k}} \mu(d) \left\lfloor \frac{n}{d^k} \right\rfloor,$$

$\sum_{i \le n} 2^{\omega(i)} = \sum_{d \le n} \mu^2(d) \left\lfloor \frac{n}{d} \right\rfloor$ (block, using the squarefree count as prefix sum), $\#\{\text{perfect squares} \le n\} = \lfloor \sqrt{n} \rfloor$.

Why $\sum d(i) = \sum \left\lfloor \frac{n}{i} \right\rfloor$: each d divides exactly $\left\lfloor \frac{n}{d} \right\rfloor$ numbers $\le n$ (swap order!).

7.2 Du's sieve ("Dujiao") — $\Phi(n)$, $M(n)$ for $n \le 10^{10}$ in $O(n^{2/3})$

Recipe. Want $F(n) = \sum_{i \le n} f(i)$. Find a helper g with $g(1) = 1$ where both G and H (prefix of $h = f * g$) are easy. Since $\sum_{i \le n}(f*g)(i) = \sum_{d \le n} g(d) F(\lfloor \frac{n}{d} \rfloor)$,

$$\boxed{F(n) = H(n) - \sum_{d=2}^{n} g(d)\, F\Big(\Big\lfloor \frac{n}{d} \Big\rfloor\Big)}$$

Block the sum, memoize F in a hash map, and sieve F directly up to $L \approx n^{2/3}$ (e.g. $L = 5 \cdot 10^6$ for $n = 10^{10}$).

f	helper g	$h = f * g$	$H(n)$	$G(n)$	why it works
φ	$\mathbf{1}$	Id	$\frac{n(n+1)}{2}$	n	$\sum_{d\mid n} \varphi(d) = n$
μ	$\mathbf{1}$	ε	1	n	$\sum_{d\mid n} \mu(d) = [n = 1]$
$i \cdot \varphi(i)$	Id	Id_2	$\frac{n(n+1)(2n+1)}{6}$	$\frac{n(n+1)}{2}$	pull-out rule
$i \cdot \mu(i)$	Id	ε	1	$\frac{n(n+1)}{2}$	pull-out rule
$i^k \varphi(i)$	Id_k	Id_{k+1}	$\sum i^{k+1}$	$\sum i^k$	pull-out rule
J_k	$\mathbf{1}$	Id_k	$\sum i^k$	n	$\sum_{d\mid n} J_k(d) = n^k$
λ	$\mathbf{1}$	$[\square]$	$\lfloor \sqrt{n} \rfloor$	n	$\sum_{d\mid n} \lambda(d) = [\square]$

The two you'll write most: $\Phi(n) = \dfrac{n(n+1)}{2} - \sum_{k=2}^{n} \Phi\Big(\Big\lfloor \frac{n}{k} \Big\rfloor\Big)$, $\quad M(n) = 1 - \sum_{k=2}^{n} M\Big(\Big\lfloor \frac{n}{k} \Big\rfloor\Big)$.

```
const int L = 5000000;                          // ~ n^(2/3); phi_pre/mu_pre = prefix sums from linear sieve
long long phiP[L + 1]; int muP[L + 1];
unordered_map<long long, long long> mpPhi; unordered_map<long long, long long> mpMu;
long long Phi(long long n) {                    // careful: n(n+1)/2 overflows past ~4e9 -> use __int128 or mod
    if (n ≤ L) return phiP[n];
    auto it = mpPhi.find(n); if (it ≠ mpPhi.end()) return it->second;
    long long res = (__int128)n * (n + 1) / 2;
    for (long long l = 2, r; l ≤ n; l = r + 1) { r = n / (n / l); res -= (r - l + 1) * Phi(n / l); }
    return mpPhi[n] = res;
}
long long Mu(long long n) {
    if (n ≤ L) return muP[n];
    auto it = mpMu.find(n); if (it ≠ mpMu.end()) return it->second;
    long long res = 1;
    for (long long l = 2, r; l ≤ n; l = r + 1) { r = n / (n / l); res -= (r - l + 1) * Mu(n / l); }
    return mpMu[n] = res;
}
```

Speed-up: instead of a hash map, store results at index `N/n` in an array (since every argument is $\lfloor N/k \rfloor$).

7.3 When Du fails: powerful numbers and Min_25

Powerful-number trick. If $f(p) = g(p)$ at every prime for a simple multiplicative g with easy prefix G, write $f = g * h$. Then $h(p) = 0$, so h is nonzero only on *powerful* numbers ($p \mid k \Rightarrow p^2 \mid k$), of which there are only $O(\sqrt{n})$. Compute

$$h(p^k) = f(p^k) - \sum_{j=1}^{k} g(p^j)\, h(p^{k-j}), \qquad F(n) = \sum_{\text{powerful } k \le n} h(k)\, G\Big(\Big\lfloor \frac{n}{k} \Big\rfloor\Big)$$

by DFS over primes $p \le \sqrt{n}$ and exponents ≥ 2.

Min_25 sieve. Sum of $f(p)$ over primes $\le n$, or full multiplicative prefix sums, in $\approx O(n^{3/4}/\log n)$. Gives $\pi(n)$ and $\sum_{p \le n} p$ for $n \le 10^{11}$ quickly. Use only when primes-at-huge-n are required.

8. GCD, LCM and Divisor Sums

Let $L = \min(n, m)$ and $S(x) = \frac{x(x+1)}{2}$.

Quantity	Formula
$\sum_{i\le n}\sum_{j\le m}\gcd(i,j)$	$\sum_{d\le L}\varphi(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$
#coprime pairs in $[1,n]\times[1,m]$	$\sum_{d\le L}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$ $(=2\Phi(n)-1$ when $n=m)$
#pairs with gcd $=k$	same as above with $n\to\left\lfloor\frac{n}{k}\right\rfloor$, $m\to\left\lfloor\frac{m}{k}\right\rfloor$
$\sum_{i\le n}\gcd(i,n)$ (Pillai)	$\sum_{d\mid n}d\,\varphi(n/d)$ (multiplicative: at p^k it is $(k+1)p^k-kp^{k-1}$)
$\sum_{i\le n}\operatorname{lcm}(i,n)$	$\frac{n}{2}\left(1+\sum_{d\mid n}d\,\varphi(d)\right)$
$\sum_{i\le n,\ \gcd(i,n)=1} i$	$\frac{n\varphi(n)}{2}$ for $n>1$; equals 1 at $n=1$
$\sum_{i\le n}\sum_{j\le m}\operatorname{lcm}(i,j)$	$\sum_{T\le L}T\Big(\sum_{d\mid T}d\,\mu(d)\Big)S(\left\lfloor\frac{n}{T}\right\rfloor)S(\left\lfloor\frac{m}{T}\right\rfloor)$
$\sum_{i\le n}\sum_{j\le m}d(ij)$	$\sum_{g\le L}\mu(g)D(\left\lfloor\frac{n}{g}\right\rfloor)D(\left\lfloor\frac{m}{g}\right\rfloor)$, $D(x)=\sum_{k\le x}\left\lfloor\frac{x}{k}\right\rfloor$
#r-tuples in $[1,M]^r$ with gcd 1	$\sum_{d\le M}\mu(d)\left\lfloor\frac{M}{d}\right\rfloor^r$
$\sum_{i<j\le n}\gcd(i,j)$ (all pairs)	$\sum_{d\le n}\varphi(d)\binom{\lfloor n/d\rfloor}{2}$

The $l=1$ / $n=1$ fixed point

$\sum_{k\le l,\gcd(k,l)=1}k=\frac{l\varphi(l)}{2}$ is **wrong at** $l=1$ (gives 1/2, true value 1). Pairing $k\leftrightarrow l-k$ breaks when $k=l-k$. Always test $n=1$.

8.1 Array gcd template

Array $a_1,\dots,a_n$ with values $\le M$. Let $c(d)=\#\{i:d\mid a_i\}$ (computed in $O(M\log M)$ from a frequency array). Then for any weight w:

$$\sum_{i,j}w(\gcd(a_i,a_j))=\sum_{d=1}^{M}(w*\mu)(d)\,c(d)^2.$$

For $w=\mathrm{Id}$, $w*\mu=\varphi$: $\sum_{i,j}\gcd(a_i,a_j)=\sum_d\varphi(d)c(d)^2$. For lcm sums use $\operatorname{lcm}(a,b)=\frac{ab}{\gcd(a,b)}$ and $c(d)$ replaced by $s(d)=\sum_{d\mid a_i}a_i$: $\sum_{i,j}\operatorname{lcm}(a_i,a_j)=\sum_d h(d)\,s(d)^2$ where $h=(\frac{1}{\mathrm{Id}})*\mu$, i.e. $h(d)=\sum_{e\mid d}\mu(d/e)/e$ (compute modulo P with inverses).

```
for (int d = 1; d ≤ M; d++) for (int k = d; k ≤ M; k += d) c[d] += freq[k];
long long ans = 0; for (int d = 1; d ≤ M; d++) ans += (long long)phi[d] * c[d] * c[d];
// ordered pairs including i=j. Unordered i<j: (ans - sum a_i) / 2
```

8.2 Counting under gcd conditions — subsequences

#non-empty subsets with gcd exactly 1: $\sum_d\mu(d)(2^{c(d)}-1)$. With gcd exactly g: subtract-downward on $f(d)=2^{c(d)}-1$.

8.3 Necklaces (Burnside + classification)

#necklaces, length n, k colors (rotation only) $=\dfrac{1}{n}\sum_{d\mid n}\varphi(n/d)\,k^d$, with exactly m ones $=\dfrac{1}{n}\sum_{d\mid\gcd(n,m)}\varphi(d)\binom{n/d}{m/d}$.

Bracelets (rotation + reflection): add $\frac{1}{2}$ of reflections: n odd: $k^{(n+1)/2}$ term; n even: $\frac{k^{n/2}+k^{n/2+1}}{2}$ term; total $=\frac{1}{2}$(necklaces) $+\frac{1}{2}$(that term).

Trap

Dividing by n under a modulus: multiply by n^{-1}, and make sure $\gcd(n,\mathrm{MOD})=1$.

9. Modular Arithmetic

9.1 Core snippets

```
long long pw(long long a, long long e, long long m) {          // a^e mod m
    long long r = 1 % m; a %= m; if (a < 0) a += m;
    for (; e; e >>= 1, a = (__int128)a * a % m) if (e & 1) r = (__int128)r * a % m;
    return r;
}
long long egcd(long long a, long long b, long long& x, long long& y) {  // ax + by = g
    if (!b) { x = 1; y = 0; return a; }
    long long x1, y1, g = egcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1; return g;
}
long long inv(long long a, long long m) {                       // needs gcd(a,m)=1, m need NOT be prime
    long long x, y; egcd((a % m + m) % m, m, x, y); return (x % m + m) % m;
}
```

```
// all inverses 1..n mod prime P in O(n)
inv[1] = 1; for (int i = 2; i <= n; i++) inv[i] = (P - P / i) * inv[P % i] % P;
// factorials + inverse factorials (nCr in O(1))
fact[0] = 1; for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % P;
ifact[n] = pw(fact[n], P - 2, P); for (int i = n; i > 0; i--) ifact[i - 1] = ifact[i] * i % P;
// nCr = fact[n] * ifact[r] % P * ifact[n - r] % P
```

Batch inverses of arbitrary $a_1..a_n$: prefix products P_i, invert P_n once, walk backwards: $a_i^{-1} = P_{i-1} \cdot (P_i)^{-1}$, then $(P_{i-1})^{-1} = (P_i)^{-1} a_i$.

9.2 Theorem table

Tool	Statement	Cost
Fermat	$a^{p-1} \equiv 1 \pmod p$ for prime $p \nmid a$. So $a^{-1} \equiv a^{p-2}$.	$O(\log p)$
Euler	$a^{\varphi(m)} \equiv 1 \pmod m$ if $\gcd(a, m) = 1$.	
Exponent reduction	$\gcd(a, m) = 1$: $a^e \equiv a^{e \bmod \varphi(m)}$.	
Euler tower (any a)	$a^e \equiv a^{(e \bmod \varphi(m)) + \varphi(m)} \pmod m$ whenever $e \geq \log_2 m$. Recurse $m \to \varphi(m)$; depth $O(\log m)$.	$O(\sqrt{m} \log m)$
Linear congruence	$ax \equiv b \pmod m$ solvable iff $g = \gcd(a, m) \mid b$; then g solutions mod m: $x_0 = \frac{b}{g} \cdot (\frac{a}{g})^{-1} \bmod \frac{m}{g}$, plus multiples of m/g.	$O(\log m)$
CRT	$x \equiv r_1 \ (m_1)$, $x \equiv r_2 \ (m_2)$ solvable iff $\gcd(m_1, m_2) \mid r_2 - r_1$; unique mod lcm.	$O(\log m)$
Wilson	$(p-1)! \equiv -1 \pmod p \iff p$ prime.	
Legendre	$v_p(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{p^k} \right\rfloor = \frac{n - s_p(n)}{p-1}$ (s_p = base-p digit sum).	$O(\log_p n)$
Kummer	$v_p \binom{n}{r}$ = number of carries when adding $r + (n-r)$ in base p.	$O(\log_p n)$
Lucas	$\binom{n}{r} \equiv \prod \binom{n_i}{r_i} \pmod p$ (digits base p). $\binom{n}{r}$ odd $\iff (r \,\&\, n) = r$.	$O(\log_p n)$
Frobenius coin	$\gcd(a, b) = 1$: largest non-representable $ab - a - b$; count of non-representable $\frac{(a-1)(b-1)}{2}$.	$O(1)$
Miller–Rabin	Bases $\{2, 3, 5, \ldots, 37\}$ (first 12 primes) deterministic for $n < 2^{64}$.	$O(12 \log n)$
Pollard Rho	Expected $O(n^{1/4})$ per factor extraction.	
BSGS	Discrete log $a^x \equiv b \pmod m$ in $O(\sqrt{m})$.	
Tonelli–Shanks	$\sqrt{a} \bmod p$.	$O(\log^2 p)$
Hensel	Lift root of $f(r) \equiv 0 \pmod{p^k}$ to p^{k+1} if $f'(r) \not\equiv 0 \pmod p$.	$O(k)$

9.3 CRT, general (non-coprime moduli)

```
// merge x = r1 (mod m1) with x = r2 (mod m2). Returns {r, lcm} or {-1,-1}.
pair<long long,long long> crt(long long r1, long long m1, long long r2, long long m2) {
    long long x, y, g = egcd(m1, m2, x, y);
    if ((r2 - r1) % g) return {-1, -1};
```

```
    long long l = m1 / g * m2;
    long long t = (__int128)((r2 - r1) / g) * x % (m2 / g);
    long long r = ((__int128)m1 * t + r1) % l; if (r < 0) r += l;
    return {r, l};
}
```

9.4 Miller–Rabin + Pollard Rho (factor up to 10^{18})

```
typedef unsigned long long ull; typedef __int128 i128;
ull mulm(ull a, ull b, ull m) { return (i128)a * b % m; }
ull pwm(ull a, ull e, ull m) { ull r = 1; a %= m; for (; e; e >>= 1, a = mulm(a, a, m)) if (e & 1) r =
    mulm(r, a, m); return r; }
bool isPrime(ull n) {
    if (n < 2) return false;
    for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) if (n % p == 0) return n == p;
    ull d = n - 1; int s = 0; while (!(d & 1)) d >>= 1, s++;
    for (ull a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        ull x = pwm(a, d, n); if (x == 1 || x == n - 1) continue;
        bool ok = false; for (int i = 1; i < s && !ok; i++) { x = mulm(x, x, n); ok = (x == n - 1); }
        if (!ok) return false;
    }
    return true;
}
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ull rho(ull n) {
    if (n % 2 == 0) return 2;
    while (true) {
        ull x = rng() % n, y = x, c = rng() % (n - 1) + 1, d = 1;
        auto f = [&](ull v) { return (mulm(v, v, n) + c) % n; };
        while (d == 1) { x = f(x); y = f(f(y)); d = __gcd(x > y ? x - y : y - x, n); }
        if (d != n) return d;
    }
}
void factorBig(ull n, vector<ull>& out) {      // unsorted primes with multiplicity
    if (n == 1) return; if (isPrime(n)) { out.push_back(n); return; }
    ull d = rho(n); factorBig(d, out); factorBig(n / d, out);
}
```

9.5 Order, primitive roots, discrete roots

- $\mathrm{ord}_m(a) \mid \varphi(m)$. To find it: $d = \varphi(m)$; for each prime $q \mid \varphi(m)$, while $q \mid d$ and $a^{d/q} \equiv 1$, set $d \leftarrow d/q$.
- Primitive root (an element of order $\varphi(m)$) exists iff $m \in \{1, 2, 4, p^k, 2p^k\}$; there are $\varphi(\varphi(m))$ of them. Find: try $g = 2, 3, \ldots$; g works iff $g^{\varphi(m)/q} \not\equiv 1$ for every prime $q \mid \varphi(m)$. Smallest is usually tiny ($\le$ a few hundred). 998244353 has primitive root 3.
- $x^k \equiv a \pmod p$ ($a \neq 0$) is solvable iff $a^{(p-1)/\gcd(k,p-1)} \equiv 1$; then it has exactly $\gcd(k, p-1)$ solutions.
- $\#\{x \bmod p : x^k \equiv 1\} = \gcd(k, p-1)$.

9.6 Quadratic residues

Legendre symbol $\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod p$ (1 if a is a nonzero square mod p, -1 if not, 0 if $p \mid a$). Exactly $\frac{p-1}{2}$ nonzero QRs. -1 is a QR iff $p \equiv 1 \pmod 4$; 2 is a QR iff $p \equiv \pm 1 \pmod 8$. Reciprocity: $\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{p-1}{2}\cdot\frac{q-1}{2}}$.

9.7 Lifting the Exponent (LTE)

For prime p, $p \nmid a$, $p \nmid b$:

- Odd p, $p \mid a - b$: $v_p(a^n - b^n) = v_p(a-b) + v_p(n)$.
- Odd p, $p \mid a + b$, n odd: $v_p(a^n + b^n) = v_p(a+b) + v_p(n)$.
- $p = 2$, a, b odd: n odd $\Rightarrow v_2(a^n - b^n) = v_2(a-b)$; n even $\Rightarrow v_2(a^n - b^n) = v_2(a-b) + v_2(a+b) + v_2(n) - 1$.

9.8 Extra medium-level modular facts

- $\sum_{i=0}^{n-1} a^i = \frac{a^n-1}{a-1}$; if $a \equiv 1 \pmod P$ this is n. If $a-1$ is not invertible, use the divide-and-conquer: $S(2k) = S(k)(1+a^k)$, $S(2k+1) = S(2k) + a^{2k}$.
- $\binom{p}{k} \equiv 0 \pmod p$ for $0 < k < p$; $(a+b)^p \equiv a^p + b^p$.
- $\binom{2n}{n} \bmod p$ for large n: Lucas. Mod composite m: factor $m = \prod p^e$, compute mod each p^e (generalized Lucas / remove p-factors from factorials), then CRT.
- Catalan $C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$ (second form avoids division).
- $\varphi(mn) = \varphi(m)\varphi(n)\frac{g}{\varphi(g)}$ with $g = \gcd(m,n)$.
- $a^{b^c} \bmod p$: compute $b^c \bmod (p-1)$ first (if $p \nmid a$). Beware $a \equiv 0$.
- The sequence $a^k \bmod m$ is eventually periodic with period dividing ord; pre-period $\le \log_2 m$.
- $x^2 \equiv 1 \pmod{p^k}$ (p odd) has exactly 2 solutions; mod 2^k ($k \ge 3$) it has 4.
- Count of $x \in [0, m)$ with $ax \equiv b$: $\gcd(a, m)$ if divisible else 0.
- Inverse of 2 mod odd P: $(P+1)/2$.

10. Combinatorial Number Theory Cheats

- **Divisor-sum swap:** $\sum_{i \le n} \sum_{d \mid i} f(d) = \sum_{d \le n} f(d) \left\lfloor \frac{n}{d} \right\rfloor$.
- **Multiples harmonic:** $\sum_{d \le N} N/d \approx N \ln N$ — loops "for d, for multiples of d" are $O(N \log N)$.
- **Lattice points visible from origin** in $[1,n]^2$: $2\Phi(n) - 1$. Including axes ($[0,n]^2 \setminus (0,0)$): $2\Phi(n) + 1$.
- **Farey sequence** F_n length: $1 + \Phi(n)$.
- **Fractions a/b with $1 \le a < b \le n$ in lowest terms**: $\Phi(n) - 1$.
- **Steps of $i \mapsto i + k \bmod n$ before returning:** $n/\gcd(n,k)$; number of cycles $\gcd(n,k)$.
- **Divisors of n that are $\equiv$ ...**: just enumerate (small).
- gcd **of a range** under updates: sparse table / segment tree; gcd changes at most $\log V$ times along a prefix — so "all distinct subarray gcds ending at i" is a list of $O(\log V)$ values.
- **Sum of $\gcd(i,j)$ over $i,j \le n$ with n up to 10^{10}:** $\sum_d \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor^2$, block with Du's Φ.
- **Number of (a,b) with $\operatorname{lcm}(a,b) = n$:** $\prod(2e_i + 1)$; with $\gcd = g, \operatorname{lcm} = l$: $2^{\omega(l/g)}$ (if $g \mid l$).
- **Number of solutions to $\frac{1}{x} + \frac{1}{y} = \frac{1}{n}$** (positive): $d(n^2)$.
- **Number of ways to write n as sum of consecutive positive integers**: number of odd divisors of n.
- **$x^2 + y^2 = n$ representations** (ordered, with signs): $4 \sum_{d \mid n,\ d \text{ odd}} (-1)^{(d-1)/2}$. Representable iff every prime $\equiv 3 \pmod 4$ has even exponent.
- **Pythagorean triples**: primitive ones are $(m^2 - k^2, 2mk, m^2 + k^2)$ with $m > k$, $\gcd(m,k) = 1$, $m - k$ odd.
- **Bezout:** $\{ax + by\}$ = multiples of $\gcd(a,b)$. Solutions: $x = x_0 + t\frac{b}{g}$, $y = y_0 - t\frac{a}{g}$. Count solutions with $x, y \ge 0$: bound t from both inequalities.

11. Sieves, Costs and Sizing

Method	Gives	Cost
Linear sieve	spf, μ, φ, primes; any multiplicative f	$O(N)$
Divisor loop	d, σ, any $\sum_{d \mid n}$	$O(N \log N)$
Segmented sieve	primes in $[L,R]$, $R \le 10^{12}$, $R - L \le 10^7$	$O((R-L)\log\log R + \sqrt{R})$
Du sieve	$\Phi(n)$, $M(n)$ for $n \le 10^{10}$–10^{11}	$O(n^{2/3})$
Min_25	prime sums, mult. prefix sums	$O(n^{3/4}/\log n)$

```cpp
// segmented sieve: primes in [L, R]
vector<bool> segSieve(long long L, long long R, const vector<int>& small) { // small primes ≤ sqrt(R)
    vector<bool> is(R - L + 1, true);
    for (long long p : small) {
        if (p * p > R) break;
        for (long long x = max(p * p, (L + p - 1) / p * p); x ≤ R; x += p) is[x - L] = false;
    }
    if (L == 1) is[0] = false;
    return is;                                  // is[i] -> L+i is prime
}
```

Sizing numbers:

- $\sum_{i\le n} d(i) \approx n \ln n$; $\sum_{i\le n} \sigma(i) \approx \frac{\pi^2}{12} n^2$; $\Phi(n) \approx \frac{3}{\pi^2} n^2 \approx 0.304 n^2$.
- squarefree $\le n \approx \frac{6}{\pi^2} n \approx 0.608 n$; $\pi(n) \approx n/\ln n$ ($\pi(10^6) = 78498$, $\pi(10^7) = 664579$, $\pi(10^9) \approx 5.08 \cdot 10^7$).
- Max $d(n)$: 240 ($n \le 10^6$), 1344 (10^9), 6720 (10^{12}), 103680 (10^{18}).
- $\omega(n) \le 9$ for $n \le 10^9$ ($2 \cdot 3 \cdots 23 = 223092870$), $\omega(n) \le 15$ for $n \le 10^{18}$.
- Prime gaps below 10^{18} are < 1500: "next prime after x" by Miller–Rabin scanning is fast.

12. Workflow and Debugging Checklist

1. **Write the brute force** as nested sums. Don't be clever yet.
2. **Expand** the blocker: a value with φ (or J_k), a condition with μ (or λ).
3. **Swap** the order of summation. Inner sum becomes $\lfloor \frac{n}{d} \rfloor$.
4. If two nested divisor sums remain, **collect on** $T = de$, identify the coefficient $h(T)$, sieve it once.
5. **Size against** n**:** $n \le 10^7$: sieve and loop. $n \le 10^{10}$: floor blocks + Du. Beyond: Min_25 / powerful numbers.
6. Prefer the **subtract-downward loop** to writing μ by hand when you need every d.
7. **Check the modulus** before `pw(a, MOD-2)` (prime?) or dividing by n in Burnside ($\gcd(n, \text{MOD}) = 1$?).
8. **Verify against brute force** on small n (1..50). Watch fixed points ($l = 1$, $n = 1$).

Classic WA sources

- `i*p` overflow inside sieve loops (cast to `long long`).
- $\frac{n(n+1)}{2}$ for $n \sim 10^{10}$ overflows `long long` after squaring further — use `__int128` or reduce mod first (and multiply by inverse of 2).
- Negative mod in C++: `((x%m)+m)%m`.
- φ values are always positive; μ prefix sums can be negative.
- `1 % m` when $m = 1$ in power function.
- Floor block with two variables: always `min` of both r's.

13. Recognize the Problem → Jump Here

You see...	Go to
"count pairs with gcd $= 1$ / $= k$"	§3 example, §8 table
"sum of gcd/lcm over a grid"	§8 table
"array, count/sum over pairs by gcd"	§5.1 subtract-downward, §8.1
"$\sum f(i) \cdot g(n/i)$" or anything with $\lfloor n/i \rfloor$	§6.1 floor blocks
many queries (n, m) up to 10^7	§6.2 collect on T + prefix sums
n up to 10^{10}, need $\sum \varphi$ or $\sum \mu$	§7.2 Du sieve
count numbers $\le X$ coprime to n	§5.3 inclusion–exclusion
squarefree / k-free counting	§7.1
factor one big number (10^{18})	§9.4 Pollard Rho
a^{b^c}, huge exponent tower	§9.2 Euler tower
system of congruences	§9.3 CRT
$\binom{n}{r}$ mod p with n huge	Lucas §9.2
power of p in $n!$ or in $\binom{n}{r}$	Legendre / Kummer §9.2
counting colorings up to rotation	§8.3 necklaces
$ax + by = c$, coin problems	Bezout §10, Frobenius §9.2